Machine Learning Assisted Adaptive Chunk Caching for IO Workloads

Review 3 technical report and experimental evidence | Sashwath J | 24BAI0059

Abstract

This project tests whether a learned cache-admission decision can reduce backing-file reads in a bounded chunk cache. A histogram gradient-boosting classifier predicts whether a requested key will reappear within the next 1,024 trace requests. On a cache miss, a native C replay reads the 4 KiB block, evaluates the frozen model, and either bypasses insertion or passes the block to ARC or a local S3-FIFO replacement policy. At a 512-block capacity, the ML-assisted variants made fewer aligned direct-read calls than their respective parent policies in all eight saved comparisons across four held-out streams. The reduction ranged from about 1.0% to 7.5%. Five-run median elapsed time improved in six comparisons and worsened in two. The supported outcome is therefore improved read-call efficiency in this prototype, with a measurable compute and timing trade-off. The result does not establish physical-device savings or a universal latency gain.

Keywords: cache admission; chunk caching; gradient boosting; direct I/O; ARC; S3-FIFO; trace replay.

1 Introduction and scope transition

The Review 2 implementation investigated CPU chunk scheduling for multiprocessing workloads. Its tests did not establish a reliable overall completion-time benefit. The final prototype keeps the larger question of learned decisions about fixed-size work units, but moves the decision to I/O cache admission: whether a fetched chunk should occupy limited cache capacity. This is a disclosed change of application and metric, not an improvement to the old scheduler. The CPU results are not combined with the I/O results below.

A cache miss requires the program to fetch a backing block. In a small cache, admitting a block unlikely to be reused may evict one that would have served a later request. The model supplies a future-reuse estimate from information available at the current request. The native cache still serves every miss correctly; the prediction changes only retention. The primary metric is the number of aligned direct-read calls made while serving a fixed trace. Elapsed time is a secondary check because model evaluation also costs CPU time.

2 Problem statement objectives and contribution

Problem statement. Given a bounded 512-entry cache of 4 KiB chunks and a request stream, can a frozen prediction of near-term reuse reduce backing-file read calls when placed before established replacement policies?

The implementation objectives are to train a model on chronological trace data without future information in online features; export it to an actual C decision path; compare each ML-assisted cache with its unchanged parent policy on held-out streams; and verify equal outputs by checksum. The project-level contribution is this measured integration and its documented transfer and timing limits. ML admission and the replacement policies themselves already have prior art, so the report does not claim invention of a new general algorithm.

3 Prior work context

CACHEUS uses learned adaptation among cache experts, which establishes that machine learning for cache replacement is not new (Rodriguez et al., 2021). S3-FIFO studies a simple three-queue replacement design, including an admission-like small queue that limits pollution by one-use objects (Yang et al., 2023). Baleen combines ML admission and prefetching for flash caches and evaluates an end-to-end backend-load metric (Wong et al., 2024). Our prototype asks a narrower engineering question: what happens when a frozen reuse

classifier filters admissions before two existing parent policies in one native direct-I/O replay? These three anchors are context, not the 30-paper analytical survey required for the student's final paper.

4 System architecture and module workflow

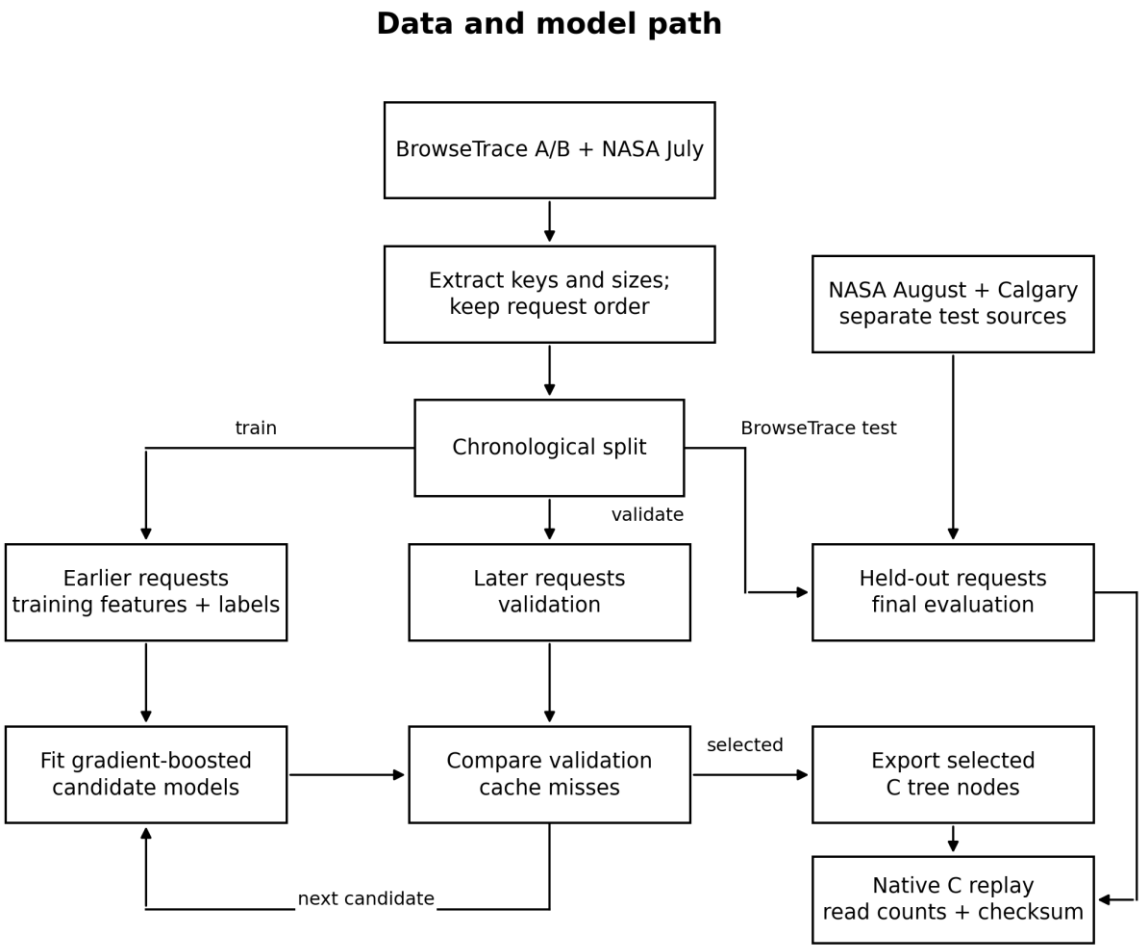


Figure 1. Chronological training, validation, model selection and held-out native replay.

BrowseTrace A and B and the earlier NASA July web log supply chronological training and validation requests. Candidate models are fitted from the earlier requests; validation miss counts select the saved configuration. Separate NASA August and Calgary web logs join the final test inputs, not the training inputs. The frozen tree nodes are exported to C before the held-out replay. This branch and feedback loop is the actual model-selection procedure, not a runtime learning loop.

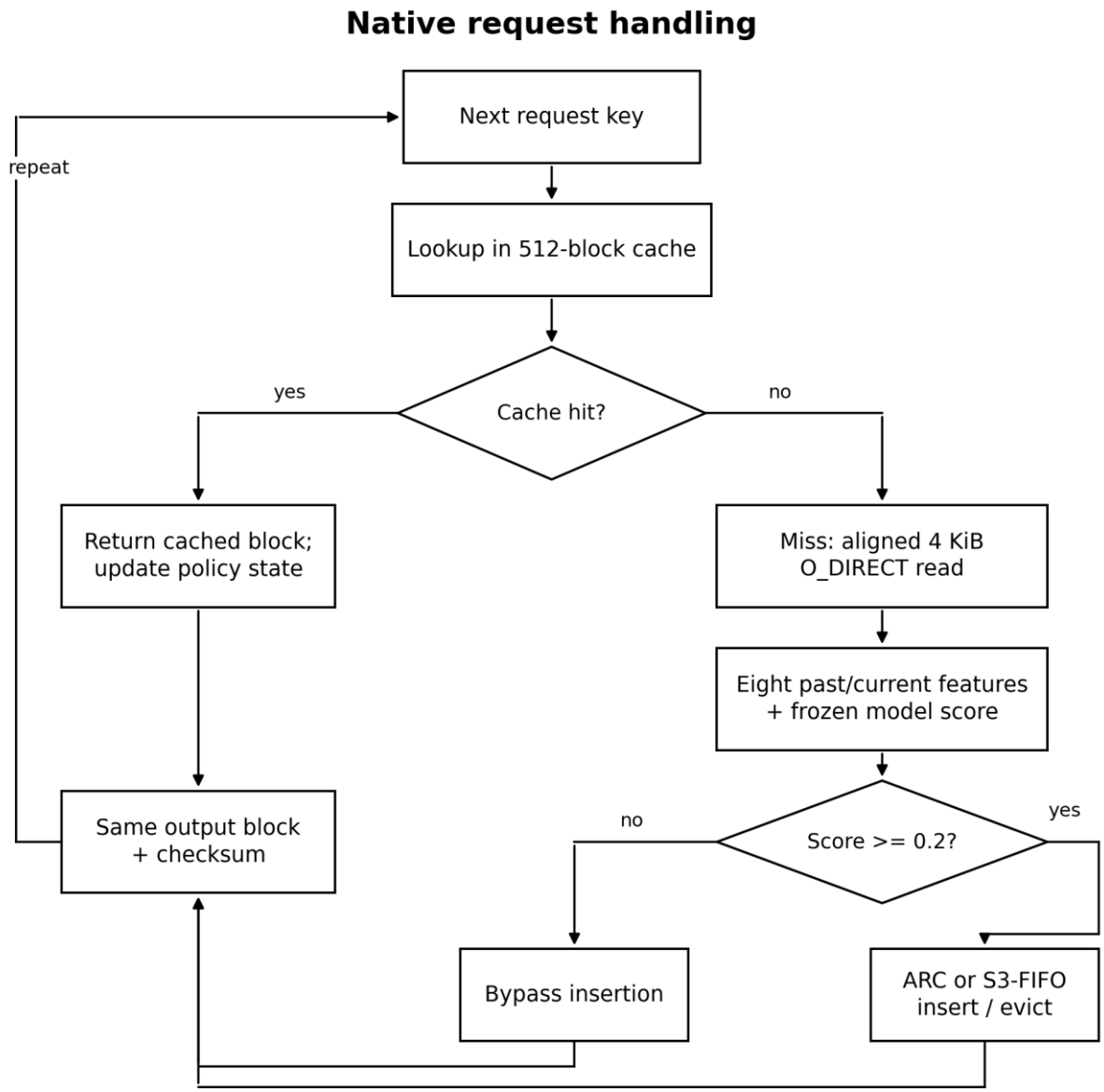


Figure 2. Cache hit, cache miss, model admission and next-request loop.

A hit returns the resident 4 KiB block without a backing read. A miss requires an aligned read; the frozen model then scores eight current-or-past features. A score below 0.2 bypasses insertion, while a score at or above 0.2 passes the block to ARC or local S3-FIFO, which may evict a resident block. Both paths return the same data and update the checksum before processing the next request. The model changes future retention, not the read needed for the current miss.

5 Data and training procedure

All sources in this report are sequences of web requests, not spacecraft measurements or operating-system sensor data. BrowseTrace A and B contribute chronological 60% training, 20% validation and final 20% test slices. Earlier requests from a 1995 NASA website access log add training and validation examples. A later August log from the same NASA website tests transfer over time. The Calgary website access log tests transfer to a different server. Only the last 100,000 valid successful GET requests from each separate test log are replayed. The training set contains 414,142 requests and the validation set contains 138,047. The reuse label is one if the same key occurs again within the next 1,024 requests in that trace slice; the label is not available to the live cache.

Source	Train	Validation	Final test
BrowseTrace A	49,473	16,491	16,491
BrowseTrace B	214,669	71,556	71,557
NASA web July	150,000	50,000	Excluded tail
NASA web August	0	0	100,000
Calgary web	0	0	100,000

Table 1. Request counts by source and use. The NASA July tail was excluded after an earlier transfer failure. The ML+S3 pairing was explored after the Calgary plain-policy result was viewed, so Calgary is not a blind selection test of that pairing.

6 Frozen model and online features

The actual fitted estimator is scikit-learn's HistGradientBoostingClassifier. It has 100 boosting stages, at most 15 leaves per tree, at least 40 samples per leaf, learning rate 0.07, L2 regularization 2 and random seed 811. The exported model contains 2,900 numerical nodes. A validation search over 40 or 100 stages and thresholds 0.2, 0.4 and 0.6 selected the saved 100-stage, 0.2-threshold configuration. No retraining takes place during native replay.

Online feature	Measured from	Purpose
Log response size	Current object metadata	Object-size signal
Log prior key count	Earlier same-key requests	Frequency
Log recency	Distance to prior same-key request	Recent locality
Log prior host count	Earlier host requests	Host activity
Path length	Current URL path	Resource structure
Static suffix	Current path suffix	Resource-type proxy
Query flag	Current URL	Dynamic URL proxy
Key length	Current full key	Key structure

Table 2. Eight online inputs. The live program does not inspect the future reuse label. The special recency value for a never-seen key uses the known replay-slice length; this is a replay convention that should become a fixed sentinel in a production service.

7 Native implementation and correctness checks

The C loop keeps at most 512 resident 4 KiB blocks, equal to 2 MiB of data capacity. A miss issues an aligned O_DIRECT read to a deterministic surrogate backing file, evaluates the exported trees and optionally inserts the block. The baseline ARC and local S3-FIFO paths use the same request order, cache capacity, block size and backing data. Each variant returns the same checksum for a given stream. Independent Python reference replays match the native miss counts, and saved evidence includes sanitizer checks. The backing blocks are surrogates because the original web response bodies are unavailable; a read call is therefore the precise measured I/O event, not a claim about production HTTP latency or physical disk transactions.

8 Evaluation metrics and comparison design

Direct-read calls count how often the native program requests a backing block; fewer calls mean better cache efficiency in this replay. Hit rate is hits divided by all requests. The per-run loop time is measured inside the native request loop. Five runs were stored for each policy and stream, and the report uses their median as a descriptive summary. The range is shown because five single-host observations are insufficient for a stable performance guarantee. A fair ML effect compares ML+ARC against ARC and ML+S3-FIFO against S3-FIFO on the same stream. Comparing the two parent algorithms alone does not isolate the learned admission decision.

Stream	ARC	ML + ARC	S3-FIFO	ML + S3-FIFO
BrowseTrace A	7,430	7,104	7,773	7,336
BrowseTrace B	17,913	17,740	18,950	18,586
NASA web	8,792	8,130	8,349	8,048
Calgary web	16,493	15,718	15,406	15,177

Table 3. Exact saved native direct-read calls. Both ML variants made fewer calls than their respective parents on each stream. The next section shows the same results visually and keeps timing separate.

9 Experimental figures and interpretation

Each figure below is generated from saved benchmark JSON or frozen-model diagnostics. The explanation immediately following it states both what the plot measures and what it cannot establish. The plotting source and tabulated values remain in the project for inspection.

9.1 Aligned direct-read calls across four held-out streams

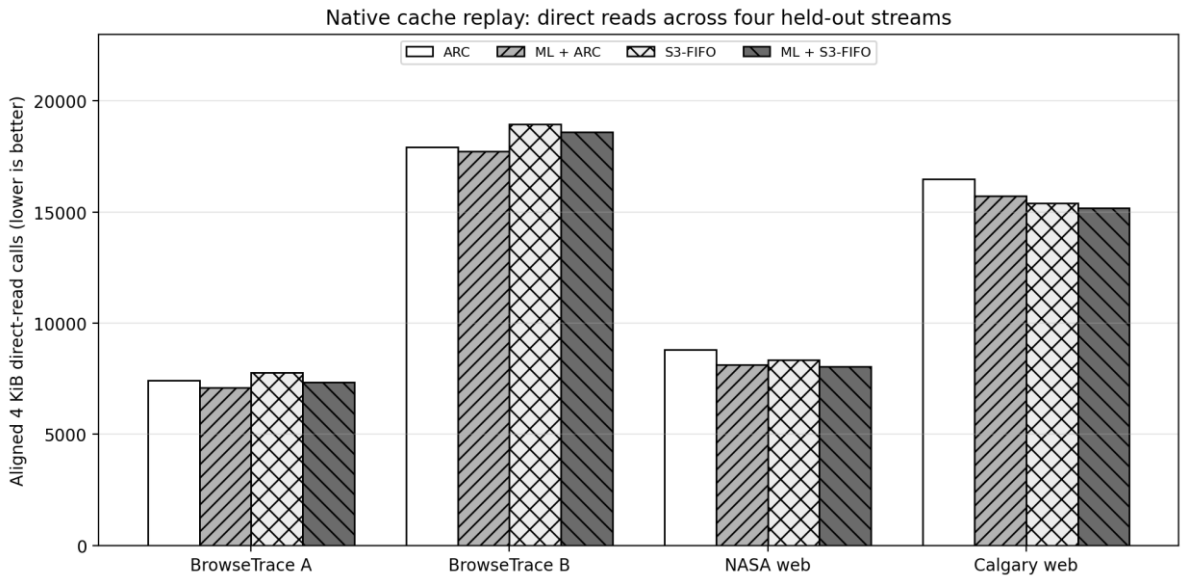


Figure 3. Aligned direct-read calls across four held-out streams.

These are absolute 4 KiB read-call counts, not inferred disk transactions. On BrowseTrace A, for example, ML+ARC makes 7,104 calls versus ARC's 7,430, while ML+S3-FIFO makes 7,336 versus its parent's 7,773. The same within-parent direction holds on all four streams. The different heights of ARC and S3-FIFO alone should not be attributed to the model.

9.2 Percentage read reduction against each parent policy

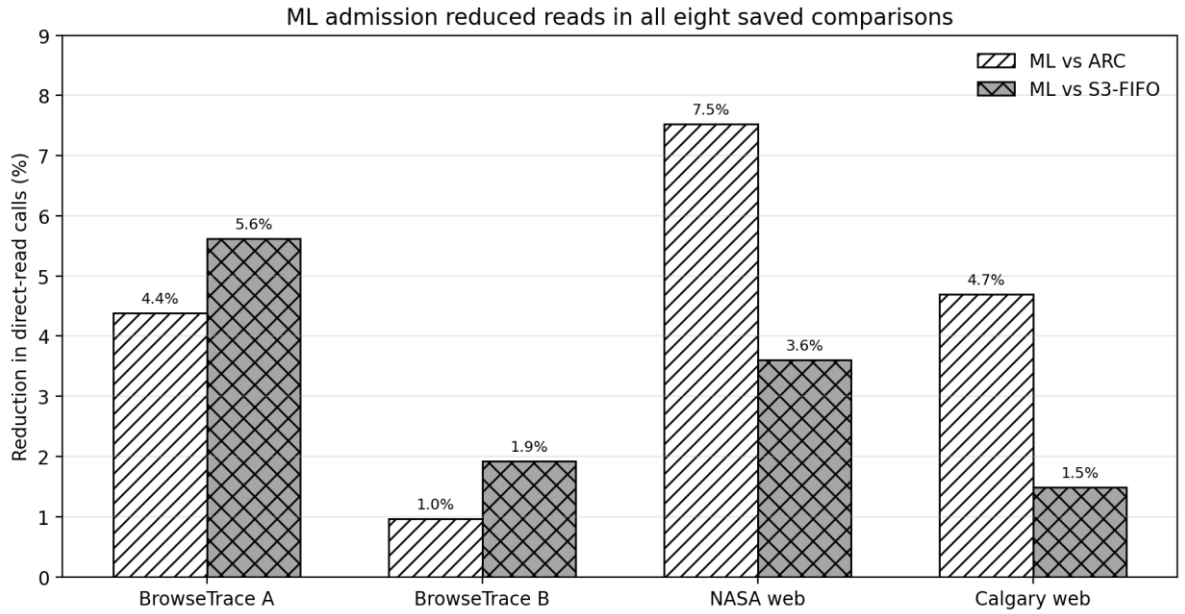


Figure 4. Percentage read reduction against each parent policy.

The percentage calculation is (parent reads - hybrid reads) / parent reads. The largest saved reduction is about 7.5% for ML+ARC on the NASA August web log; the smallest is about 1.0% for ML+ARC on BrowseTrace B. Every bar is positive, but the size of the effect changes with the request stream and replacement policy. This supports a modest, workload-dependent I/O-efficiency benefit.

9.3 All five saved timing observations for each policy and stream

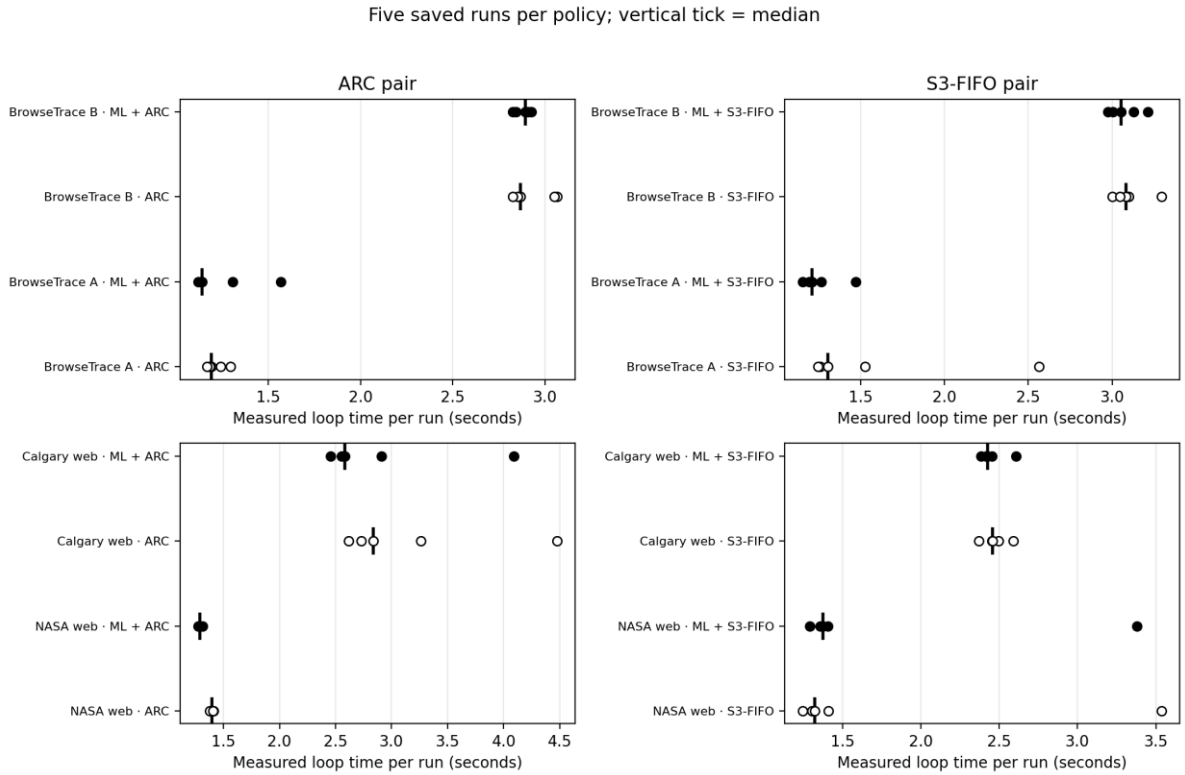


Figure 5. All five saved timing observations for each policy and stream.

Dots are individual native loop times and the short vertical mark is their median. The wide spread in several groups, especially Calgary-web ARC and NASA-web S3-FIFO, shows why one fast run would be misleading. These points are not confidence intervals and were not collected on multiple hosts. The precise direct-read counts repeat, while wall time is sensitive to execution conditions.

9.4 Change in five-run median loop time after adding ML admission

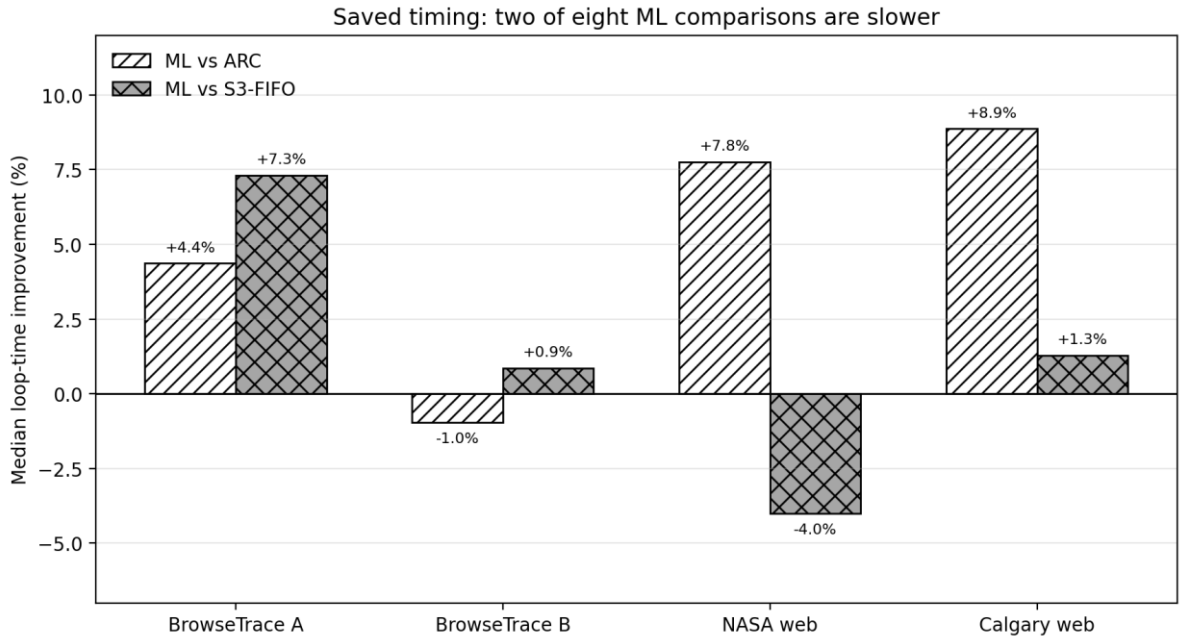


Figure 6. Change in five-run median loop time after adding ML admission.

Six of eight bars lie above zero, meaning the stored median loop time decreased. Two lie below: ML+ARC on BrowseTrace B and ML+S3-FIFO on the NASA August web log. The latter is about 4% slower despite fewer reads. The experiment does not isolate model-evaluation CPU time from all other timing variation, so the chart supports a trade-off observation, not a proven causal breakdown.

9.5 Read-call savings and elapsed-time change in the same comparison

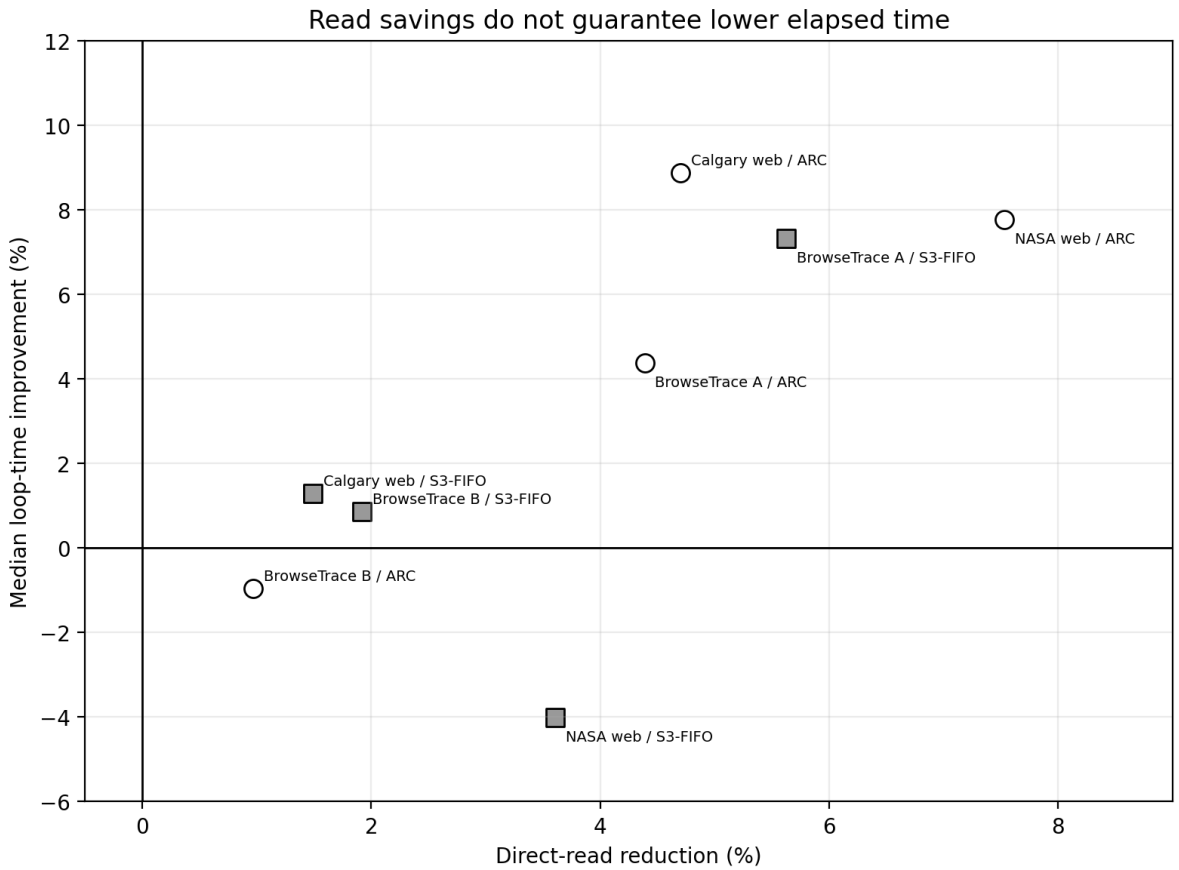


Figure 7. Read-call savings and elapsed-time change in the same comparison.

Points to the right save backing reads. Points above zero also have lower median loop time. Two points are right of zero but below zero, showing that fewer read calls need not make a run faster. The case for this system is reducing expensive read requests when that objective matters; it cannot be sold as an unconditional speedup.

9.6 Measured cache-hit rates for four policies on four streams

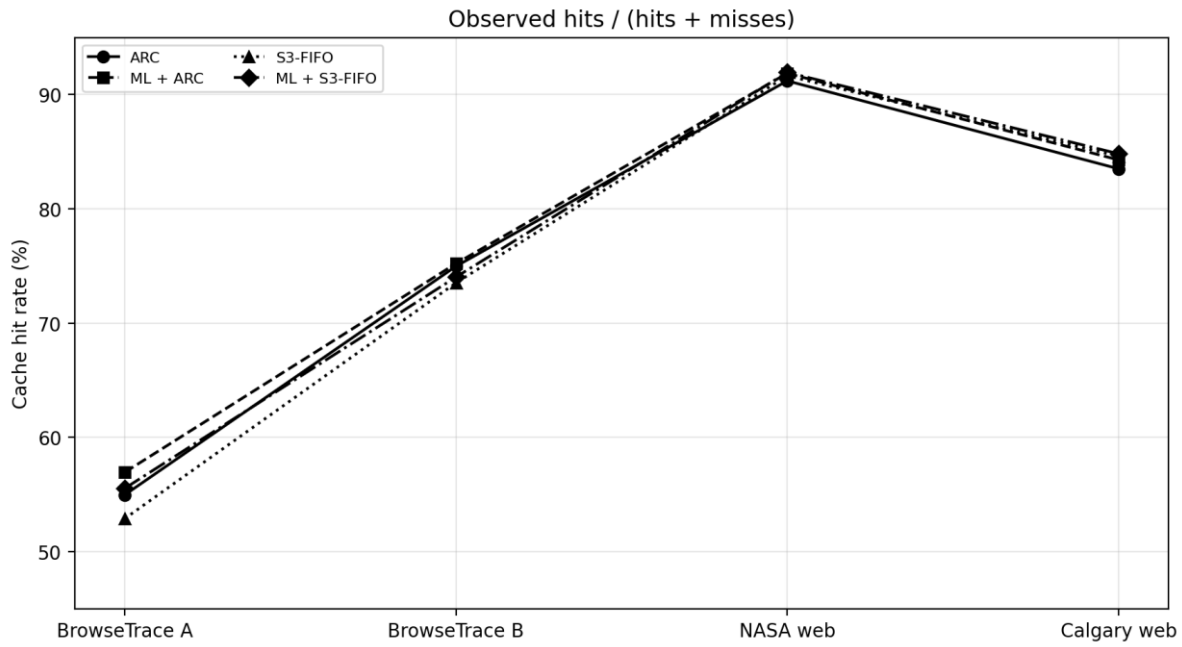


Figure 8. Measured cache-hit rates for four policies on four streams.

Hit rate equals recorded hits divided by hits plus direct-read misses. The ML-filtered version has a slightly higher hit rate than its parent in each saved pairing, consistent with the read counts. The figure also shows that baseline difficulty differs by stream: the NASA August web log has much higher hit rates than BrowseTrace A, so identical percentage gains should not be expected.

9.7 Held-out ranking quality and target prevalence for the frozen model

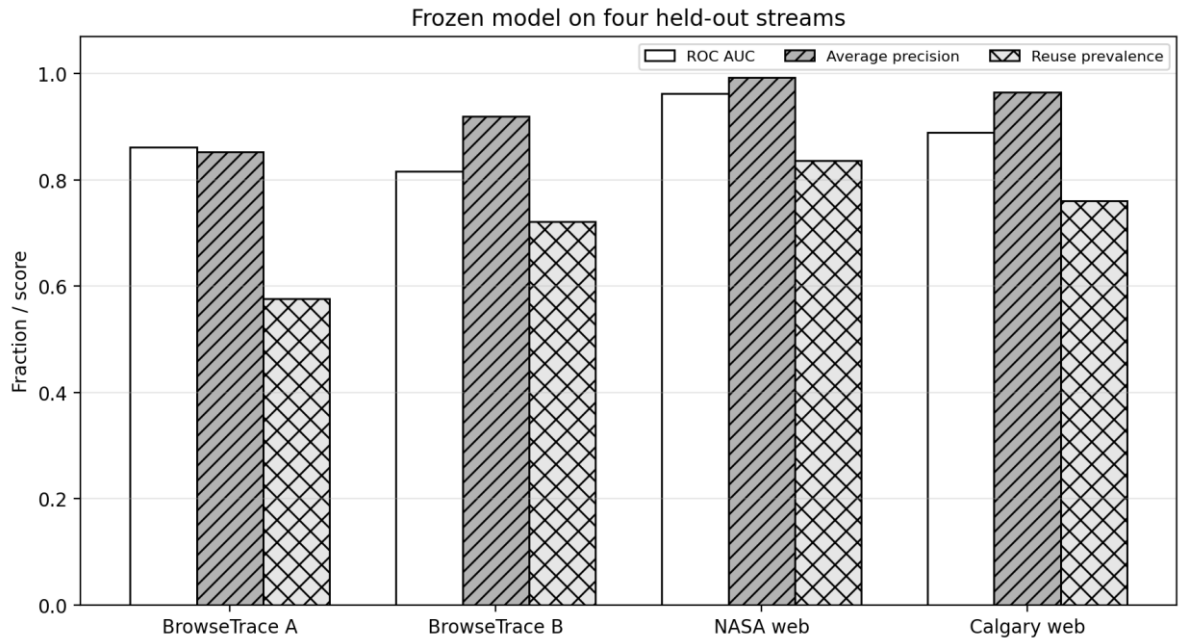


Figure 9. Held-out ranking quality and target prevalence for the frozen model.

ROC AUC ranges from about 0.816 on BrowseTrace B to 0.962 on the NASA August web log. Average precision must be read alongside prevalence because reuse is common in these slices. A high classifier score describes discrimination of the defined 1,024-request reuse label; it does not by itself prove a cache-level read saving. That claim comes from the native replay.

9.8 Precision and recall at the operational 0.2 threshold

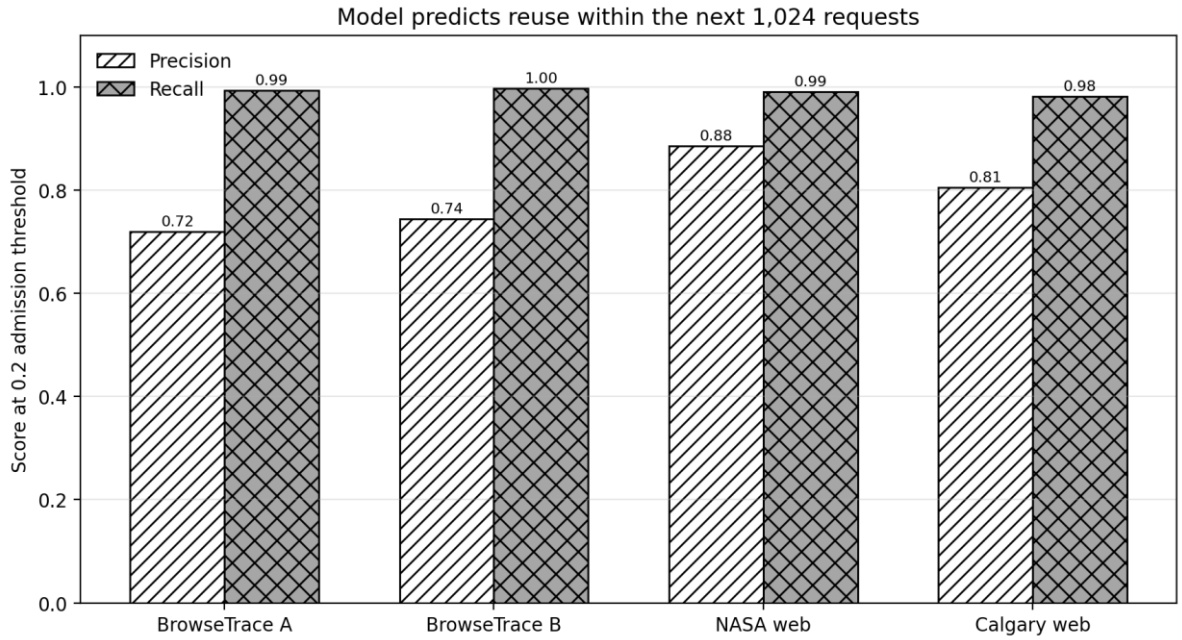


Figure 10. Precision and recall at the operational 0.2 threshold.

The permissive threshold retains nearly all positive reuse labels, with recall about 0.982 to 0.997. Precision is lower, ranging about 0.720 to 0.885. In plain terms, the model rarely rejects a chunk that will reappear in the defined horizon, but it still admits some chunks that do not. These values use offline future labels only for evaluation.

9.9 Classification outcomes as a proportion of each held-out stream

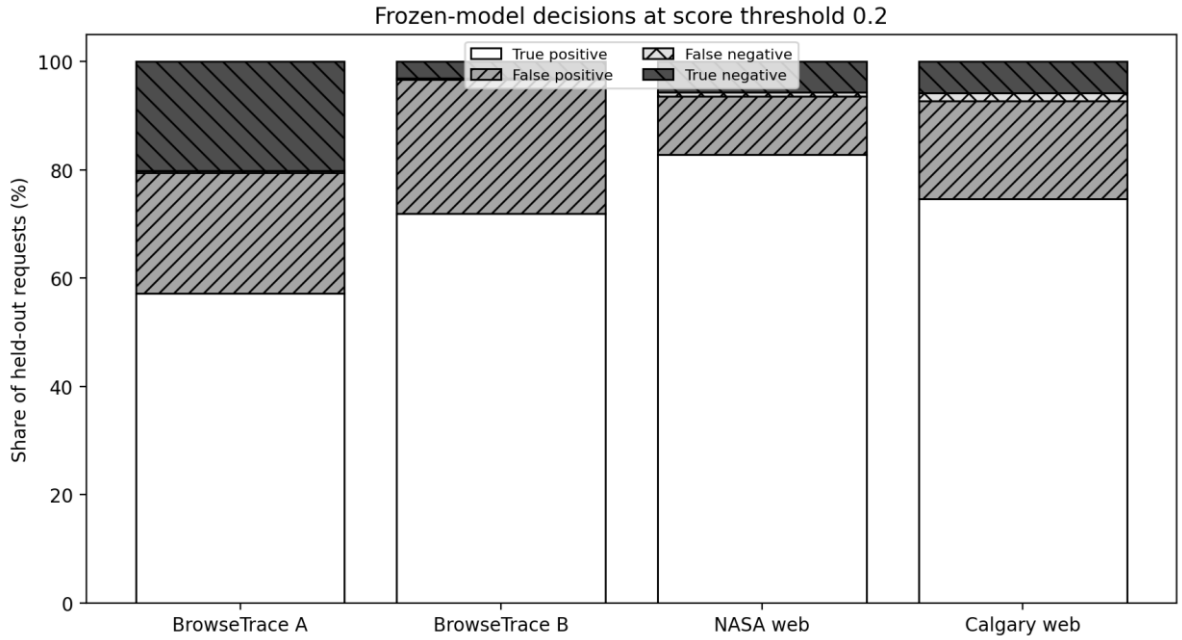


Figure 11. Classification outcomes as a proportion of each held-out stream.

The false-positive portion represents admitted blocks that do not reappear within the labeling horizon. The false-negative portion is small because the threshold favors recall. Class prevalence differs by stream, so raw TP and FP proportions should not be compared without the positive-share context in Figure 9. The finite trace end right-censors some labels.

9.10 Cumulative misses at the first three recorded checkpoints

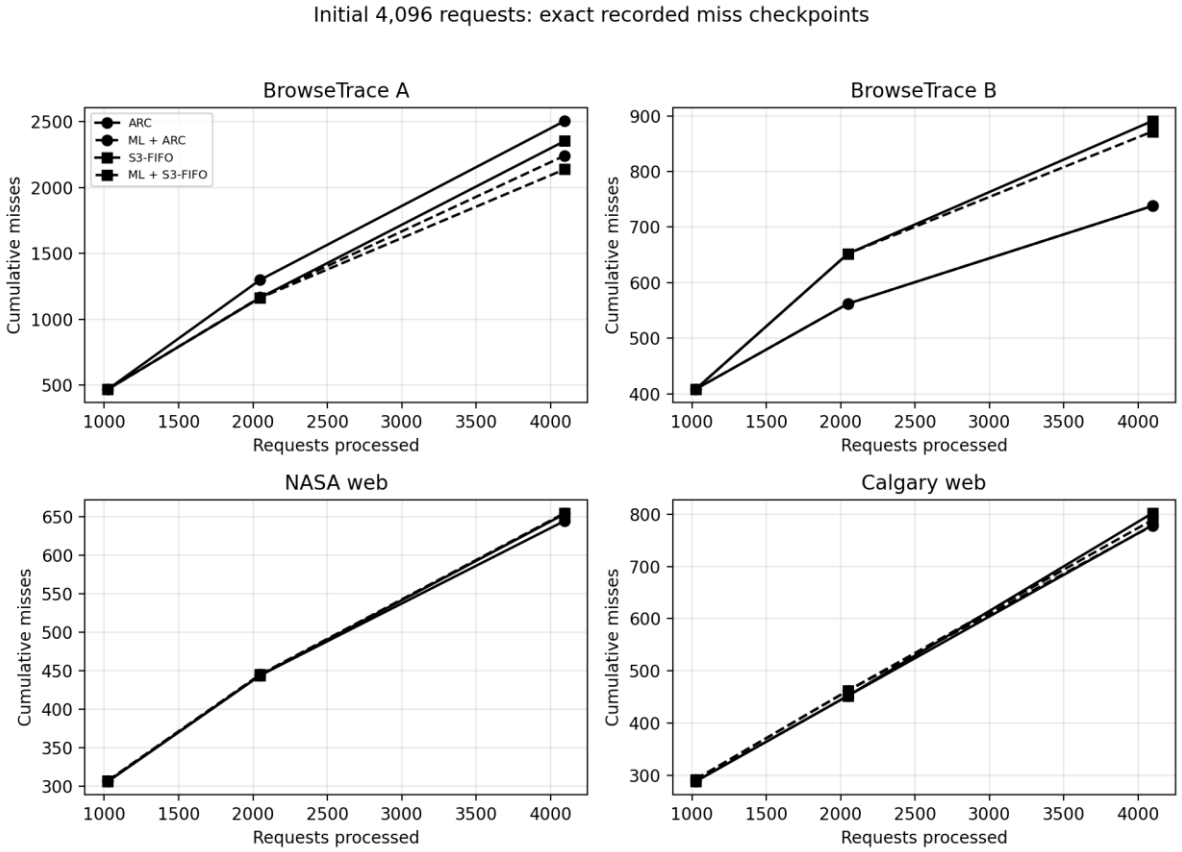


Figure 12. Cumulative misses at the first three recorded checkpoints.

These exact checkpoints occur after 1,024, 2,048 and 4,096 requests. The line segments connect only those observations; they are not a continuous trace of every intervening miss. Early differences are sometimes small even when the full-stream totals diverge. This figure therefore documents warm-up behavior but cannot replace the complete-count comparison in Figure 3.

10 Model calibration and interpretation

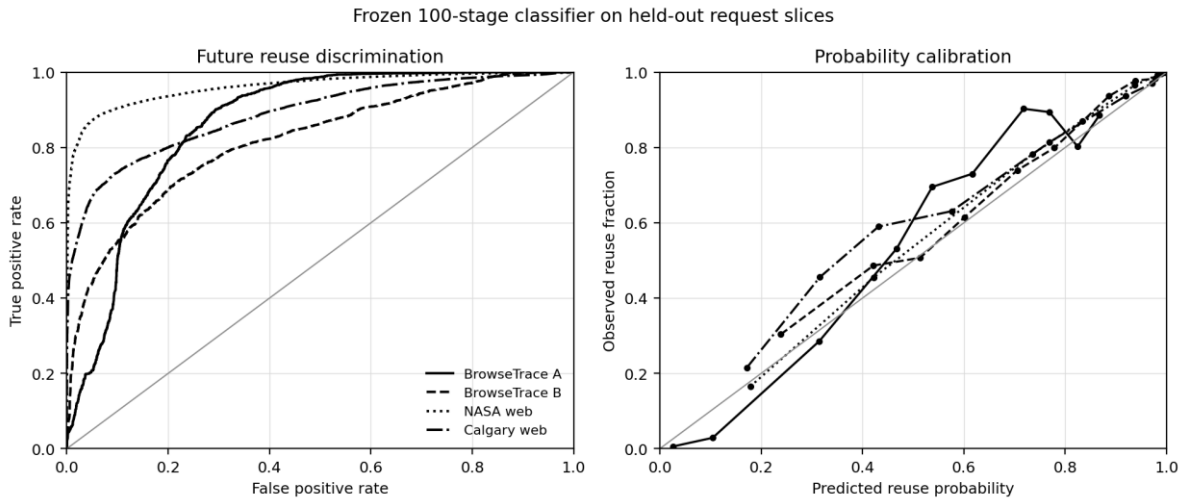


Figure 13. ROC and calibration curves recomputed from the saved model on final test slices.

The ROC panel tests whether higher scores rank future-reuse examples above non-reuse examples. The calibration panel compares predicted probability with observed frequency. The curves differ by source, which is evidence of distribution shift and imperfect probability calibration. The admission threshold was chosen on validation data, not retuned on these held-out slices. The Calgary web log is not fully blind for the later ML+S3 pairing, as stated earlier.

Test stream	ROC AUC	Precision	Recall	Brier
BrowseTrace A	.862	.720	.993	.143
BrowseTrace B	.816	.744	.997	.150
NASA web	.962	.885	.991	.058
Calgary web	.889	.805	.982	.118

Table 4. Frozen-model diagnostics at threshold 0.2 except ROC AUC and Brier, which use scores over all thresholds. These are classifier measures, not direct system metrics.

11 Interpretation limitations and remaining research

The evidence supports fewer native direct-read calls with learned admission on the four tested request streams, under a fixed 512-block cache and deterministic 4 KiB backing data. It does not show that physical device I/O, energy, wear, network latency or multiprocess throughput improved. The measured request loop is single-process and user-space; its S3-FIFO branch is local rather than the official optimized implementation. The cache traces have finite-horizon labels and source shifts. Calgary was inspected before the later S3 pairing. Five timing repeats on one machine should be treated as indicative only. Larger caches, different chunk sizes and genuinely concurrent shared-cache workloads would be follow-up experiments, not claims of completed work.

12 Demonstration and reproducibility

From the project directory, `python demo_hybrid_final.py` checks saved source hashes, model settings, read counts and policy checksums, then prints the four-stream comparison. `python demo_hybrid_final.py --live calgary` recompiles and runs a fresh native Linux direct-I/O replay through WSL and GCC. Fresh counts and checksums should match the saved result; elapsed times may differ. `python verify_hybrid_native.py`

independently checks the native policies against Python reference replays. The plotting command is `.venv\Scripts\python.exe plot_review_gallery.py` on the Windows project host.

The inspectable code paths are `train_three_source_cache.py` for training, `export_cache_native.py` for frozen model export, `cache_native.c` for online reads and admission, `benchmark_hybrid_native.py` for stored repeats, `evaluate_frozen_model.py` for classifier diagnostics, and `plot_review_gallery.py` for the numbered result figures. The JSON inputs and the exported model are retained alongside them. Load the trusted local `.joblib` model only; pickle-family model formats can execute code during loading.

13 Conclusion

The prototype uses a real trained model in the native miss path and produces a small but consistent reduction in backing-file read calls against each of its two parent cache policies on four saved streams. The gain is not free: the program must evaluate the model, and two stored comparisons have worse median elapsed time. The defensible final result is improved cache-read efficiency for these traces and this capacity, with the implementation and failure cases available for inspection. It is a scope change from Review 2 CPU chunk scheduling, not a claim that the original scheduling problem was solved.

References

Rodriguez, L. V., Yusuf, F., Lyons, S., Paz, E., Rangaswami, R., Liu, J., Zhao, M., & Narasimhan, G. (2021). Learning cache replacement with CACHEUS. In 19th USENIX Conference on File and Storage Technologies (pp. 341-354). USENIX Association. https://www.usenix.org/conference/fast21/presentation/rodriguez

Wong, D. L.-K., Wu, H., Molder, C., Gunasekar, S., Lu, J., Khandkar, S., Sharma, A., Berger, D. S., Beckmann, N., & Ganger, G. R. (2024). Baleen: ML admission and prefetching for flash caches. In 22nd USENIX Conference on File and Storage Technologies (pp. 347-371). USENIX Association. https://www.usenix.org/conference/fast24/presentation/wong

Yang, J., Zhang, Y., Qiu, Z., Yue, Y., & Rashmi, K. V. (2023). FIFO queues are all you need for cache eviction. In Proceedings of the 29th Symposium on Operating Systems Principles. Association for Computing Machinery. https://github.com/Thesys-lab/sosp23-s3fifo

Academic submission status

This is an illustrated technical evidence companion for review and student editing. The supplied Review 3 guideline requires an original student-written paper with at least 30 relevant peer-reviewed sources from 2021-2026, analytical comparison and fully matched APA citations. This document has three verified context sources and does not meet that paper requirement by itself. The student must complete and verify the literature work, write the submission in their own words, and confirm the changed scope with the reviewer before claiming final academic compliance.